PROJECT & OPERATIONAL KNOWLEDGE

Blameless Incident Postmortem: SEV-1 Database Pool Exhaustion

Outage Timeline, Technical Root Cause, 5 Whys Analysis, and Preventive Action Items

Severity: SEV-1 High | **Duration:** 42 Minutes | **Incident Commander:** Alex Mercer (Staff SRE) | **Impacted Users:** ~4,200 Internal Users

1. Incident Overview and Metadata

This blameless postmortem documents the SEV-1 production incident on August 14, 2026, affecting the Enterprise Knowledge Assistant (EKA) API cluster. Grounded in Google SRE postmortem best practices, this review identifies root causes, lessons learned, and preventive action items.

Attribute	Details
Incident Identifier	INC-84920 (SEV-1 Critical Availability Impairment)
Incident Title	Database Connection Pool Exhaustion on Knowledge Assistant API
Date & Duration	August 14, 2026 | 14:18 UTC – 15:00 UTC (42 minutes total duration)
Impact	98.4% of API queries failed with HTTP 500/504 errors; ~4,200 internal users affected; zero data corruption.
Incident Commander	Alex Mercer (Staff SRE) | Comms Lead: Sarah Jenkins

2. Chronological Incident Timeline (August 14, 2026)

- **14:18 UTC:** Scheduled Celery batch worker begins bulk ingestion of 15,000 engineering wiki documents.
- **14:22 UTC:** Celery autoscaling launches 128 parallel worker containers. Workers connect directly to PostgreSQL, bypassing the PgBouncer connection pooler due to a configuration drift in `celery_settings.py`.
- **14:25 UTC:** PostgreSQL hits `max_connections = 500`. All database connections saturate. API service requests queue and fail.
- **14:27 UTC:** Datadog synthetic probe triggers PagerDuty alert: *"EKA Query API 5xx rate > 15%"*.
- **14:31 UTC:** Incident Commander Alex Mercer declares SEV-1; war room assembled on Slack `#incident-84920`.
- **14:36 UTC:** Database DBA identifies 480 connections originating from Celery worker IP range.
- **14:41 UTC:** IC issues mitigation command: `kubectl scale deployment/celery-ingestion --replicas=0`.
- **14:46 UTC:** Active connections plummet to 45. API p99 latency normalizes to 180ms. Health checks return green.
- **15:00 UTC:** 15-minute monitoring soak window concludes with 0 errors. Incident declared resolved.

3. Root Cause Analysis (5 Whys)

1. **Why did the API fail?** PostgreSQL database rejected new connections, causing HTTP 500 errors.
2. **Why did the database reject connections?** The active connection limit (500) was fully saturated.
3. **Why was the connection limit saturated?** 128 Celery worker pods opened direct persistent database connections.
4. **Why did Celery bypass the connection pooler?** A recent PR updated the database host to the raw RDS endpoint rather than the PgBouncer internal DNS.
5. **Why did CI not catch this?** Staging tests only execute against a single shared container without connection limits.

4. Corrective and Preventive Action Items

Action Item	Owner	Target Date	Status
Hardcode PgBouncer pooler endpoint into Helm charts	DevOps (R. Patel)	Aug 18, 2026	Completed
Implement PgBouncer max client connection caps per user	DBA (K. Vance)	Aug 22, 2026	In Progress
Add Datadog alert at 75% connection pool saturation	SRE (A. Mercer)	Aug 19, 2026	Completed